Sequential Task:
Logistics System

Object-Oriented Programming and GUI

March 29, 2026

1. Unpack the archive attached to the assignment and place its contents in your IDE project. Make sure that the package structure remains unchanged.
2. Run the program. The output should be:

   ```
   === CARGO - Logistics System ===
   ```

3. In the package `task05`, create a subpackage named `container`.
4. In the subpackage `container`, create the file `Box.java` and declare the generic class `Box<T>`, which will represent a container storing exactly one element of any type.
   (a) Add a private field `content` of type `T`, which stores the current contents of the container.
   (b) Implement the no-argument constructor `Box()`, whose purpose is to create an empty container.
   - Set the field `content` to `null`.

   (c) Implement the constructor `Box(T content)`, which initializes the container with the provided contents.
   - Accept the parameter `content`, representing the initial element.
   - Assign the passed value to the field `content`.

   (d) Implement the method `put(T item)`, whose purpose is to place an element into the container.
   - Check whether the field `content` is not equal to `null`.
   - If the container is not empty, throw an `IllegalStateException` with a message stating that the container must be emptied before it can be reused.
   - If the container is empty, assign the provided element `item` to the field `content`.

   (e) Implement the method `get()`, whose purpose is to retrieve the element from the container.
   - Check whether the field `content` is equal to `null`.
   - If the container is empty, throw an `IllegalStateException` with a message stating that the container is empty.

- If the container holds an element, return the value of the field `content`.

(f) Implement the method `getAndClear()`, whose purpose is to retrieve the element and simultaneously empty the container.

- Call the method `get()` and store the result in a helper variable `item`.
- Call the method `clear()` to remove the container contents.
- Return the stored value `item`.

(g) Implement the method `isEmpty()`, whose purpose is to check whether the container is empty.

- Return the boolean value resulting from the comparison `content == null`.

(h) Implement the method `clear()`, whose purpose is to remove the contents of the container.

- Set the field `content` to `null`.

(i) Override the method `toString()` so that it returns a readable representation of the container.

- Check whether the container is empty by calling `isEmpty()`.
- If it is empty, return a string containing the information `EMPTY`.
- Otherwise, use the `toString()` method of the stored element.
- Build the resulting string in a format containing the class name and the current contents.

5. In the class `Main`, find the marker `TODO 01` and uncomment the code below it. Import the class `Box` from the package `task05.container`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true
```

6. In the package `task05.container`, create the file `Pair.java`. In this file, declare the generic class `Pair<K, V>`, which represents an immutable pair of two values of different types.

(a) Add a private, immutable field `key` of type `K`, which stores the first value of the pair.

(b) Add a private, immutable field `value` of type `V`, which stores the second value of the pair.

(c) Implement the constructor `Pair(K key, V value)`, whose purpose is to initialize both values of the pair.

- Accept the parameter `key`, representing the first value.
- Accept the parameter `value`, representing the second value.
- Assign the provided values to the corresponding fields `key` and `value`.

(d) Add the method `getKey()`, which returns the value of the field `key`.

(e) Add the method `getValue()`, which returns the value of the field `value`.

(f) Implement the method `swap()`, whose purpose is to create a new pair with swapped types and values.

- Create a new object of type `Pair<V, K>`.
- Pass the current value of the field `value` as the first element and `key` as the second element to the constructor.
- Return the newly created pair.

(g) Override the method `toString()` so that it returns a readable textual representation of the pair.

- Build a string containing the class name and the values of the fields `key` and `value`.
- Use a format that indicates the relationship between the values, for example `key -> value`.
- Return the created string as the method result.

7. In the class `Main`, find the marker `TODO 02` and uncomment the code below it. Import all classes from the package `task05.container`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}
```

8. 🚩 Milestone 1: present the result of your work to the instructor

9. In the package `task05.processing`, create the file `InspectionUtils.java`. Then declare the class `InspectionUtils`, which will provide static generic helper methods for logging inspections, labeling objects, and transferring elements between containers.

(a) Implement the public static generic method `logInspection(T item)`, whose purpose is to display basic information about the provided object.

- Define the generic type parameter `<T>` so that the method can accept objects of any type.
- Accept the parameter `item`, representing the object being inspected.
- Display the inspection log section header in the console using `System.out.println`.
- Display the type name of the object by calling `item.getClass().getSimpleName()`.
- Display the text representation of the object using the value `item`, which causes its `toString()` method to be used.
- Display the bottom line closing the log section.

(b) Implement the public static generic method `label(String id, T item)`, whose purpose is to create a label connecting an identifier with the provided object.

- Define the generic type parameter `<T>` so that the method can handle objects of any type.
- Accept the parameter `id`, representing the label identifier.
- Accept the parameter `item`, representing the object to be labeled.
- Display a console message informing that the label has been assigned, using `String.format` to build a message containing the identifier and the representation of the object.
- Create a new object of type `Pair<String, T>`, passing `id` as the key and `item` as the value.
- Return the created pair as the method result.

(c) Implement the public static generic method `transferBetweenBoxes(Box<T> source, Box<T> destination)`, whose purpose is to move an element from one container to another.

- Define the generic type parameter `<T>` so that both containers store elements of the same type.
- Accept the parameter `source`, representing the source container.
- Accept the parameter `destination`, representing the destination container.
- Retrieve the element from the source container by calling `source.getAndClear()` and store the result in the variable `item`.
- Place the retrieved element into the destination container by calling `destination.put(item)`.
- Display a console message informing that the object has been moved.
  - Include the text representation of the moved element stored in the variable `item` in the message.

10. In the class `Main`, find the marker `TODO 03` and uncomment the code below it. Import all classes from the package `task05.processing`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===
```

```
=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}

=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}
```

11. In the package `task05.processing`, create the file `Transformer.java`. Declare the functional interface `Transformer<T, R>`, which will define a contract for objects that transform input data of one type into a result of another type.

    (a) Mark the interface with the annotation `@FunctionalInterface` to indicate that it contains exactly one method without a body and can be used with lambda expressions and method references.

    (b) Define two generic type parameters: `T` as the input type and `R` as the result type of the transformation.

    (c) Add the method `transform(T input)`, whose purpose is to transform the provided input object into a result of the appropriate type.

    - Accept the parameter `input`, representing the input data to be transformed.
    - Set the return type of the method to `R`.

- Leave the method implementation to lambda expressions, method references, or classes implementing the interface.

12. In the package `task05.processing`, create the file `PackagingTransformer.java`. Declare the class `PackagingTransformer`, which will provide ready-made data transformation implementations for different object types, as well as helper methods generating transformers.

    (a) Add the public static final field `ELECTRONICS_TO_LABEL` of type `Transformer<Electronics, String>`, which defines how an object of type `Electronics` is transformed into a text label.
    - Implement the transformation using a lambda expression accepting the parameter `e`.
    - Build the resulting string containing the device name, its voltage, and its weight.
    - Retrieve the data by calling the methods `getName()`, `getVoltage()`, and `getWeight()`.

    (b) Add the public static final field `FOOD_WEIGHT_TO_GRAMS` of type `Transformer<Food, Double>`, which transforms the weight of a food product into grams.
    - Implement the transformation using a lambda expression accepting the parameter `f`.
    - Retrieve the product weight by calling `getWeight()`.
    - Multiply the value by `1000.0` to obtain the result in grams.
    - Return the calculated value of type `Double`.

    (c) Implement the public static generic method `toStringTransformer()`, which returns a transformer converting any object to its text representation.
    - Define the generic type parameter `<T>`.
    - Return an implementation of `Transformer<T, String>` using the method reference `Object::toString`.

13. In the class `Main`, find the marker `TODO 04` and uncomment the code below it. Run the program and check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}
```

```
=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true
```

14. In the package `task05.container`, create the file `Wrapper.java`. Declare the generic class `Wrapper<T>`, which represents a package storing an element together with a security seal identifier.

    (a) Add the private immutable (`final`) field `content` of type `T`, which stores the package contents.

    (b) Add the private immutable field `sealId` of type `String`, which stores the package seal identifier.

    (c) Implement the constructor `Wrapper(T content, String sealId)`, whose purpose is to initialize both fields of the class.

    - Accept the parameter `content`, representing the package contents.
    - Accept the parameter `sealId`, representing the security seal identifier.
    - Assign the provided values to the corresponding fields `content` and `sealId`.

    (d) Implement the method `unwrap()`, whose purpose is to reveal the contents of the package and return the stored element.

    - Display a message informing that the seal has been broken, using the value of the field `sealId`.

- Return the value of the field `content`.

(e) Implement the generic method `map(Transformer<T, R> transformer)`, whose purpose is to transform the package contents into a new type and return a new package.

- Define the generic type parameter `<R>`, representing the result type of the transformation.
- Accept the parameter `transformer`, which implements a transformation from type `T` to type `R`.
- Call the method `transform(this.content)` on the provided `transformer` object and store the result in the variable `transformed`.
- Create a new object of type `Wrapper<R>`, passing the transformed value and a modified seal identifier.
  - Append the suffix `"-MOD"` to the existing `sealId`.
- Return the newly created object as the method result.

(f) Add the method `getSealId()`, which returns the value of the field `sealId`.

(g) Override the method `toString()` so that it returns a readable representation of the package.

- Build a string containing the security seal identifier and the text representation of the contents.
- Use the values of the fields `sealId` and `content`.
- Return the created string as the method result.

15. In the class `Main`, find the marker `TODO 05` and uncomment the code below it. Run the program and check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}

=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
```

```
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true

=== 5. WRAPPER<T> + map() ===
Wrapper{seal='SEAL-A1', content=Electronics{name='Laptop', weight=2.5, voltage=220V}}
Transformed (still sealed): Wrapper{seal='SEAL-A1-MOD', content=[ELECTRONICS]
Laptop | 220V | 2.5kg}
Voltage check result: Wrapper{seal='SEAL-A1-MOD-MOD', content=true}
[SECURITY] Seal SEAL-A1-MOD-MOD broken.
Unwrapped value: true
```

16. 🏁 Milestone 2: present the result of your work to the instructor

17. In the package `task05.shipping`, create the file `Shipment.java`. Declare the generic class `Shipment<T extends Product>`, which represents a shipment containing cargo that is a product, together with additional logistics information.

    (a) Specify the generic type bound `T` so that it extends the class `Product`, ensuring that every cargo item has product properties.

    (b) Add the private immutable field `cargo` of type `T`, which stores the transported product.

    (c) Add the private immutable field `destination` of type `String`, which stores the shipment destination.

    (d) Add the private immutable field `priorityOverride` of type `int`, which specifies the shipment priority.

    (e) Add the private immutable field `fragile` of type `boolean`, which indicates whether the shipment is fragile.

(f) Add the private immutable field `insured` of type `boolean`, which indicates whether the shipment is insured.

(g) Implement the constructor `Shipment(T cargo, String destination, int priorityOverride, boolean fragile, boolean insured)`, whose purpose is to initialize all object fields.

- The parameter `cargo`, representing the transported product.
- The parameter `destination`, representing the destination.
- The parameter `priorityOverride`, representing the shipment priority.
- The parameter `fragile`, indicating whether the shipment is fragile.
- The parameter `insured`, indicating whether the shipment is insured.
- Assign all provided values to the corresponding class fields.

(h) Add the method `getCargo()`, which returns the value of the field `cargo`.

(i) Add the method `getDestination()`, which returns the value of the field `destination`.

(j) Override the method `toString()` so that it returns a detailed textual representation of the shipment.

- Use the method `String.format` to build a string containing information about the cargo, destination, priority, fragility status, and insurance status.
- Use the values of all class fields in the appropriate parts of the format.
- Return the created string as the method result.

18. In the package `task05.shipping`, create the file `ShipmentBuilder.java`. Declare the generic class `ShipmentBuilder<T extends Product>`, which enables step-by-step construction of a `Shipment` object using the Builder pattern.

(a) Specify the generic type bound `T` so that it extends the class `Product`, ensuring access to product properties.

(b) Add the private immutable field `cargo` of type `T`, which stores the product assigned to the shipment.

(c) Add the private field `destination` of type `String`, which stores the shipment destination, and initialize it with the default value `"UNKNOWN"`.

(d) Add the private field `priorityOverride` of type `int`, which stores the overridden priority, and initialize it with the value `-1`.

(e) Add the private field `fragile` of type `boolean`, which indicates whether the shipment is fragile, and initialize it with the value `false`.

(f) Add the private field `insured` of type `boolean`, which indicates whether the shipment is insured, and initialize it with the value `false`.

(g) Implement the constructor `ShipmentBuilder(T cargo)`, whose purpose is to set the initial shipment product.

i. Accept the parameter `cargo`, representing the product.

ii. Assign the provided value to the field `cargo`.

(h) Implement the method `to(String destination)`, whose purpose is to set the shipment destination.

- Assign the provided value to the field `destination`.
- Return the current object `ShipmentBuilder<T>` in order to enable method chaining.

(i) Implement the method `withPriority(int priority)`, whose purpose is to set a custom shipment priority.

- Assign the provided value to the field `priorityOverride`.
- Return the current object `ShipmentBuilder<T>`.

(j) Implement the method `fragile()`, whose purpose is to mark the shipment as fragile.

- Set the field `fragile` to `true`.
- Return the current object `ShipmentBuilder<T>`.

(k) Implement the method `insured()`, whose purpose is to mark the shipment as insured.

- Set the field `insured` to `true`.
- Return the current object `ShipmentBuilder<T>`.

(l) Implement the method `build()`, whose purpose is to create a complete `Shipment` object.

i. Compute the final shipment priority and store it in the variable `finalPriority`.
   - If the value of the field `priorityOverride` is greater than or equal to `0`, use that value.
   - Otherwise, retrieve the default priority by calling `cargo.getPriority()`.

ii. Create a new object of type `Shipment<T>`, passing `cargo`, `destination`, `finalPriority`, `fragile`, and `insured`.

iii. Return the created object as the method result.

19. In the class `Main`, find the marker `TODO 06` and uncomment the code below it. Import all classes from the package `task05.shipping`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}
```

```
=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true

=== 5. WRAPPER<T> + map() ===
Wrapper{seal='SEAL-A1', content=Electronics{name='Laptop', weight=2.5, voltage=220V}}
Transformed (still sealed): Wrapper{seal='SEAL-A1-MOD', content=[ELECTRONICS]
Laptop | 220V | 2.5kg}
Voltage check result: Wrapper{seal='SEAL-A1-MOD-MOD', content=true}
[SECURITY] Seal SEAL-A1-MOD-MOD broken.
Unwrapped value: true

=== 6. BUILDER<T> ===
Shipment{cargo=Electronics{name='Laptop', weight=2.5, voltage=220V}, dest=
'Warsaw, Poland', priority=1, fragile=true, insured=true}
Shipment{cargo=Food{name='Apples', weight=5.0, expires in 14 days}, dest=
'Berlin, Germany', priority=2, fragile=false, insured=false}
```

20. In the package `task05.sorting`, create the file `FieldComparator.java`. Declare the

generic class `FieldComparator<T>`, which implements the interface `Comparator<T>` and allows objects to be compared based on a selected field.

(a) Add the private immutable field `fieldExtractor` of type `Function<T, ? extends Comparable>`, which is responsible for extracting the field value used for comparison.

(b) Add the private immutable field `ascending` of type `boolean`, which determines the sort direction.

(c) Implement the constructor `FieldComparator(Function<T, ? extends Comparable> fieldExtractor, boolean ascending)`, whose purpose is to initialize the class fields.

i. Accept the parameter `fieldExtractor`, representing the function that extracts a field value from an object.

ii. Accept the parameter `ascending`, which determines the sort direction.

iii. Assign the provided values to the corresponding class fields.

(d) Override the method `compare(T o1, T o2)`, whose purpose is to compare two objects based on the selected field.

i. Call `fieldExtractor.apply(o1)` and store the result in the variable `val1`.

ii. Call `fieldExtractor.apply(o2)` and store the result in the variable `val2`.

iii. Compare the values `val1` and `val2` using the method `compareTo`.

iv. Store the comparison result in the variable `result`.

v. Return the result unchanged if the sorting order is ascending.

vi. Return the opposite value, that is, the negation of the result, if the sorting order is descending.

(e) Add the public static generic method `ascending(Function<T, U> extractor)`, which creates a comparator that sorts in ascending order.

- Restrict the type `U` to `Comparable<U>`.
- Create a new object of type `FieldComparator` with the provided extractor and ascending order.
- Return the created comparator.

(f) Add the public static generic method `descending(Function<T, U> extractor)`, which creates a comparator that sorts in descending order.

- Restrict the type `U` to `Comparable<U>`.
- Create a new object of type `FieldComparator` with the provided extractor and descending order.
- Return the created comparator.

21. In the package `task05.sorting`, create the file `SortingLine.java`. Declare the class `SortingLine`, which will provide static generic operations related to sorting and displaying arrays of elements.

(a) Implement the public static generic method `sort(T[] items, FieldComparator<T> comparator)`, whose purpose is to sort the provided array using the given comparator.

i. Define the generic type parameter `<T>` so that the method can handle arrays of objects of any type.

ii. Accept the parameter `items`, representing the array of elements to be sorted.

iii. Accept the parameter `comparator`, representing the object used to compare elements of type `T`.

iv. Implement the `Bubble Sort` algorithm.

- Run an outer `for` loop iterating from index `0` to `items.length - 2`.
- In each iteration, run an inner `for` loop iterating from index `0` to `items.length - 2 - i`.
- For each pair of adjacent elements, call the method `comparator.compare(items[j], items[j + 1])`.
- Check whether the comparison result is greater than `0`, which means that the elements are in the wrong order.
- If the condition is satisfied, swap the two elements.
  - Store the element `items[j]` in the helper variable `temp`.
  - Assign the element `items[j + 1]` to the position `items[j]`.
  - Assign the value of the variable `temp` to the position `items[j + 1]`.

(b) Implement the public static generic method `printLine(String label, T[] items)`, whose purpose is to display a section label and all elements of the provided array.

i. Define the generic type parameter `<T>` so that the method can handle arrays of objects of any type.

ii. Accept the parameter `label`, representing the name of the displayed section.

iii. Accept the parameter `items`, representing the array of elements to be printed.

iv. Display the section header as text containing the value of the parameter `label`.

v. Iterate through all elements of the array `items` using an enhanced `for-each` loop.

vi. For each element, display its text representation on a separate line.

- Prefix the printed value with a short indentation.
- Let the object's `toString()` method be used.

22. In the class `Main`, find the marker `TODO 07` and uncomment the code below it. Import all classes from the package `task05.sorting`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
```

```
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}

=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true

=== 5. WRAPPER<T> + map() ===
Wrapper{seal='SEAL-A1', content=Electronics{name='Laptop', weight=2.5, voltage=220V}}
Transformed (still sealed): Wrapper{seal='SEAL-A1-MOD', content=[ELECTRONICS]
Laptop | 220V | 2.5kg}
Voltage check result: Wrapper{seal='SEAL-A1-MOD-MOD', content=true}
[SECURITY] Seal SEAL-A1-MOD-MOD broken.
Unwrapped value: true
```

```
=== 6. BUILDER<T> ===
Shipment{cargo=Electronics{name='Laptop', weight=2.5, voltage=220V}, dest=
'Warsaw, Poland', priority=1, fragile=true, insured=true}
Shipment{cargo=Food{name='Apples', weight=5.0, expires in 14 days}, dest=
'Berlin, Germany', priority=2, fragile=false, insured=false}

=== 7. FIELD COMPARATOR<T> ===
--- Before sorting ---
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
--- Sorted by WEIGHT (asc) ---
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
--- Sorted by PRIORITY (desc) ---
  Food{name='Milk', weight=1.0, expires in 3 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
--- Sorted by NAME (asc) ---
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
```

23. 🏁 Milestone 3: present the result of your work to the instructor

24. In the package `task05.transfer`, create the file `DockTransfer.java`. Declare the class `DockTransfer`, which will provide static generic operations for transferring and copying elements between containers using wildcards.

    (a) Implement the public static generic method `transfer(Box<? extends T> source, Box<? super T> destination)`, whose purpose is to move an element from the source container to the destination container.

        i. Define the generic type parameter `<T>`.

ii. Accept the parameter `source` of type `Box<? extends T>`, which allows reading elements of type `T` or its subtypes.

iii. Accept the parameter `destination` of type `Box<? super T>`, which allows writing elements of type `T` or its supertypes.

iv. Verify that the source container is not empty by calling `source.isEmpty()`.

v. If the container is empty, throw an `IllegalStateException` with a message stating that there is no element to transfer.

vi. Retrieve the element from the source container by calling `source.getAndClear()` and store it in the variable `item`.

vii. Place the element in the destination container by calling `destination.put(item)`.

viii. Display a message informing that the element has been transferred, using `System.out.println`.

(b) Implement the public static generic method `copy(Box<? extends T> source, Box<? super T> destination)`, whose purpose is to copy an element from the source container to the destination container without removing it from the source.

i. Define the generic type parameter `<T>`.

ii. Accept the parameter `source` of type `Box<? extends T>`.

iii. Accept the parameter `destination` of type `Box<? super T>`.

iv. Verify that the source container is not empty by calling `source.isEmpty()`.

v. If the container is empty, throw an `IllegalStateException` with a message stating that there is no element to copy.

vi. Retrieve the element from the source container by calling `source.get()` and store it in the variable `item`.

vii. Place the element in the destination container by calling `destination.put(item)`.

viii. Display a message informing that the element has been copied using `System.out.println`.

25. In the class `Main`, find the marker `TODO 08` and uncomment the code below it. Import all classes from the package `task05.transfer`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}
```

```
=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true

=== 5. WRAPPER<T> + map() ===
Wrapper{seal='SEAL-A1', content=Electronics{name='Laptop', weight=2.5, voltage=220V}}
Transformed (still sealed): Wrapper{seal='SEAL-A1-MOD', content=[ELECTRONICS]
Laptop | 220V | 2.5kg}
Voltage check result: Wrapper{seal='SEAL-A1-MOD-MOD', content=true}
[SECURITY] Seal SEAL-A1-MOD-MOD broken.
Unwrapped value: true

=== 6. BUILDER<T> ===
Shipment{cargo=Electronics{name='Laptop', weight=2.5, voltage=220V}, dest=
'Warsaw, Poland', priority=1, fragile=true, insured=true}
Shipment{cargo=Food{name='Apples', weight=5.0, expires in 14 days}, dest=
'Berlin, Germany', priority=2, fragile=false, insured=false}

=== 7. FIELD COMPARATOR<T> ===
```

```
--- Before sorting ---
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
--- Sorted by WEIGHT (asc) ---
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
--- Sorted by PRIORITY (desc) ---
  Food{name='Milk', weight=1.0, expires in 3 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
--- Sorted by NAME (asc) ---
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}

=== 8. PECS (Producer Extends, Consumer Super) ===
[DOCK] Transferred: Electronics{name='Laptop', weight=2.5, voltage=220V}
General box now contains: Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
```

26. In the class `Main`, find the marker `TODO 09`. Below it, declare the static inner class `TemperatureController`, which represents a simple object storing a temperature value.

    (a) Add the private immutable field `tempCelsius` of type `double`, which stores the temperature expressed in degrees Celsius.

    (b) Implement the constructor `TemperatureController(double temp)`, whose purpose is to initialize the temperature field.

        i. Accept the parameter `temp`, representing the temperature value.

        ii. Assign the provided value to the field `tempCelsius`.

    (c) Override the method `toString()` so that it returns a readable textual representation of the object.

        - Build a string containing the class name and the current temperature value with the unit `C`.

- Use the value of the field `tempCelsius`.
- Return the created string as the method result.

27. In the class `Main`, find the marker `TODO 10`. Below it, declare the static inner class `SecurityProtocol`, which represents a security level assigned to a system or operation.

    (a) Add the private immutable field `level` of type `String`, which stores the name of the security level.

    (b) Implement the constructor `SecurityProtocol(String level)`, whose purpose is to initialize the field storing the security level.

        i. Accept the parameter `level`, representing the security level.

        ii. Assign the provided value to the field `level`.

    (c) Override the method `toString()` so that it returns a readable textual representation of the object.

        - Build a string containing the class name and the current value of the field `level`.
        - Use a format indicating the security level, for example `level=...`.
        - Return the created string as the method result.

28. In the package `task05.registry`, create the file `WarehouseRegistry.java` and declare the class `WarehouseRegistry`, which will serve as a registry storing different modules associated with their types.

    (a) Add the private immutable field `modules` of type `Map<Class<?>, Object>`, which stores registered instances associated with their classes.

    (b) Initialize the map as a `HashMap`, enabling dynamic insertion and lookup of elements.

    (c) Implement the public generic method `put(Class<T> type, T instance)`, whose purpose is to register a new module.

        i. Define the generic type parameter `<T>`.

        ii. Accept the parameter `type`, representing the module class.

        iii. Accept the parameter `instance`, representing the module instance.

        iv. Add an entry to the map `modules`, using `type` as the key and `instance` as the value.

        v. Display a message informing that the module has been registered, using `type.getSimpleName()`.

    (d) Implement the public generic method `get(Class<T> type)`, whose purpose is to retrieve a registered module based on its type.

        i. Define the generic type parameter `<T>`.

        ii. Accept the parameter `type`, representing the module class.

        iii. Retrieve the value from the map `modules` and store it in the variable `value`.

        iv. Check whether the retrieved value is equal to `null`.

        v. If no module is found, throw an `IllegalArgumentException` with a message containing the type name.

vi. Otherwise, cast the value to type `T` using the method `type.cast(value)`.

vii. Return the cast object as the method result.

(e) Implement the method `contains(Class<?> type)`, whose purpose is to check whether a given module type has been registered.

- Call the method `modules.containsKey(type)`.
- Return the result as a boolean value.

(f) Implement the method `size()`, whose purpose is to return the number of registered modules.

- Call the method `modules.size()`.
- Return the obtained value.

29. In the class `Main`, find the marker `TODO 11` and uncomment the code below it. Import all classes from the package `task05.registry`, then run the program. Check whether the output is:

```
=== CARGO - Logistics System ===

=== 1. BOX<T> ===
Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}
Box{Food{name='Apples', weight=5.0, expires in 14 days}}
Retrieved: Food{name='Apples', weight=5.0, expires in 14 days}
Box empty? true

=== 2. PAIR<K, V> ===
Pair{PKG-001 -> Electronics{name='Laptop', weight=2.5, voltage=220V}}
Swapped: Pair{Electronics{name='Laptop', weight=2.5, voltage=220V} -> PKG-001}

=== 3. GENERIC METHODS ===
=== INSPECTION LOG ===
  Type  : Electronics
  Value : Electronics{name='Laptop', weight=2.5, voltage=220V}
======================
=== INSPECTION LOG ===
  Type  : Food
  Value : Food{name='Apples', weight=5.0, expires in 14 days}
======================
=== INSPECTION LOG ===
  Type  : String
  Value : Random string on the belt
======================
[LABELER] Assigning label 'FOOD-042' to: Food{name='Apples', weight=5.0, expires
in 14 days}
```

```
Pair{FOOD-042 -> Food{name='Apples', weight=5.0, expires in 14 days}}
[TRANSFER] Moved: Electronics{name='Phone', weight=0.2, voltage=5V}
Source: Box{EMPTY}
Dest:   Box{Electronics{name='Phone', weight=0.2, voltage=5V}}

=== 4. TRANSFORMER<T, R> ===
Label: [ELECTRONICS] Laptop | 220V | 2.5kg
Apples weight: 5000.0g
Hazard: HazardousMaterial{name='Sulfuric Acid', class=Class 8}
Laptop high voltage? true

=== 5. WRAPPER<T> + map() ===
Wrapper{seal='SEAL-A1', content=Electronics{name='Laptop', weight=2.5, voltage=220V}}
Transformed (still sealed): Wrapper{seal='SEAL-A1-MOD', content=[ELECTRONICS]
Laptop | 220V | 2.5kg}
Voltage check result: Wrapper{seal='SEAL-A1-MOD-MOD', content=true}
[SECURITY] Seal SEAL-A1-MOD-MOD broken.
Unwrapped value: true

=== 6. BUILDER<T> ===
Shipment{cargo=Electronics{name='Laptop', weight=2.5, voltage=220V}, dest=
'Warsaw, Poland', priority=1, fragile=true, insured=true}
Shipment{cargo=Food{name='Apples', weight=5.0, expires in 14 days}, dest=
'Berlin, Germany', priority=2, fragile=false, insured=false}

=== 7. FIELD COMPARATOR<T> ===
--- Before sorting ---
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
--- Sorted by WEIGHT (asc) ---
  Electronics{name='Phone', weight=0.2, voltage=5V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Apples', weight=5.0, expires in 14 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
--- Sorted by PRIORITY (desc) ---
  Food{name='Milk', weight=1.0, expires in 3 days}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
```

```
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
--- Sorted by NAME (asc) ---
  Food{name='Apples', weight=5.0, expires in 14 days}
  Electronics{name='Laptop', weight=2.5, voltage=220V}
  Food{name='Milk', weight=1.0, expires in 3 days}
  Electronics{name='Phone', weight=0.2, voltage=5V}
  HazardousMaterial{name='Sulfuric Acid', class=Class 8}

=== 8. PECS (Producer Extends, Consumer Super) ===
[DOCK] Transferred: Electronics{name='Laptop', weight=2.5, voltage=220V}
General box now contains: Box{Electronics{name='Laptop', weight=2.5, voltage=220V}}

=== 9. TYPESAFE HETEROGENEOUS CONTAINER ===
[REGISTRY] Registered: TemperatureController
[REGISTRY] Registered: SecurityProtocol
Temperature module: TempController{-18.5C}
Security module:    Security{level=LEVEL-5}
Registry size:      2
```

30. ★ Milestone 4 present the result of your work to the instructor